

Stokes glacier models with Firedrake

a finite element tutorial

Ed Bueler

University of Alaska Fairbanks

version 2.4 (October 2026)

github.com/bueler/stokes-ice-tutorial



making this presentation effective

- please ask questions, thanks!
- or later questions/comments:
 - by email to elbueler@alaska.edu
 - or by using [issues at github.com/bueler/stokes-ice-tutorial/](https://github.com/bueler/stokes-ice-tutorial/issues)
- these slides document some Python codes ... please try the codes and give me your feedback?



<i>theory</i>	equations for the glacier dynamics problem
stage1/	first demo: linear Stokes
stage2/	power-law Stokes for glacier ice
stage3/	practical power: extruded meshes and 3D glaciers
<i>theory</i>	evolving glaciers: free-surface equation coupled to Stokes
stage4/	moving-margin dome solutions
	summary and learning more

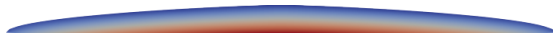
glacier dynamics using a Stokes model: the high-level view

- we apply conservation principles to glaciers and ice sheets:
 - mass conservation ✓
 - momentum conservation ✓
 - energy conservation (not here)
- incompressibility is one aspect of mass conservation
 - another is the free-surface equation ... see stage4/
- glaciers are a “slow flow” problem because inertial forces are tiny
 - momentum conservation thus called a “stress balance”
 - velocity & pressure are **instantaneously determined** by glacier geometry and boundary stresses

speed $|u|$



pressure p



- Stokes equations = stress balance + incompressibility



George Stokes

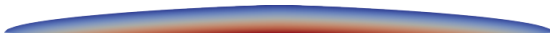
glacier dynamics using a Stokes model: the high-level view

- we apply conservation principles to glaciers and ice sheets:
 - mass conservation ✓
 - momentum conservation ✓
 - energy conservation (not here)
- incompressibility is one aspect of mass conservation
 - another is the free-surface equation ... see `stage4/`
- glaciers are a “slow flow” problem because inertial forces are tiny
 - momentum conservation thus called a “stress balance”
 - velocity & pressure are **instantaneously determined** by glacier geometry and boundary stresses

speed $|\mathbf{u}|$



pressure p



- Stokes equations = stress balance + incompressibility



George Stokes

Glen-Steinemann-Nye-Stokes equations

- canonical glacier dynamics equations: Stokes flow, with power-law viscosity from Glen, Steinemann, and Nye (1950s)
- the model here is **isotropic power-law Stokes**:

$$\begin{aligned}-\nabla \cdot \boldsymbol{\tau} + \nabla p &= \rho \mathbf{g} && \text{stress balance} \\ \nabla \cdot \mathbf{u} &= 0 && \text{incompressibility} \\ \boldsymbol{\tau} &= B_n |\mathbf{Du}|^{(1/n)-1} \mathbf{Du} && \text{flow law}\end{aligned}$$

- $\mathbf{u}$ is velocity, p is pressure
- $\boldsymbol{\tau}$ is deviatoric stress tensor
- $\mathbf{Du} = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^T)$ is strain-rate tensor
- flow law is in *viscosity form*: $\boldsymbol{\tau} = 2\nu(|\mathbf{Du}|)\mathbf{Du}$

[▶ details](#) viscosity is regularized (" ν_ϵ ") to avoid ∞ when $\mathbf{Du} \rightarrow 0$

- constants (*defaults*):

- ice density $\rho = 910 \text{ kg m}^{-3}$
- power-law exponent $n = 3$
- ice hardness $B_n = 6.8 \times 10^7 \text{ Pa s}^{1/3}$ $\leftarrow$ isothermal here



John Glen

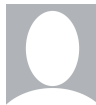


photo not available

Samuel Steinemann



John Nye

boundary conditions, as simple as possible

- note that $\sigma = 2\nu_\epsilon(|D\mathbf{u}|)D\mathbf{u} - pI$ is the **Cauchy stress tensor**
- stress-free top:

$$\sigma \mathbf{n} = (2\nu_\epsilon(|D\mathbf{u}|)D\mathbf{u} - pI) \mathbf{n} = \mathbf{0}$$

- atmospheric pressure could be applied at surface . . . no interesting effect
- no-slip base:

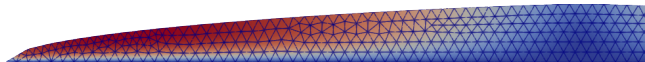
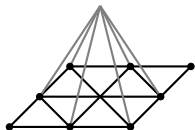
$$\mathbf{u} = \mathbf{0}$$

- **no attempt to model basal sliding** in this tutorial



finite elements ... with no details

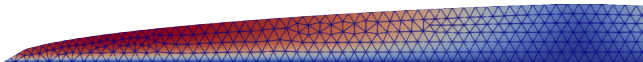
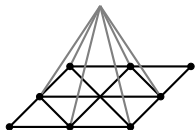
- by-hand solution of the Stokes model is impossible for most geometries
 - *exercise* on the exception: solve slab-on-a-slope
- so we solve numerically using the finite element (FE) method
 - this tutorial **suppresses the details** of FE
 - try e.g. Elman, Sylvester & Wathen (2014) for FE self-study
 - ▶ details one-slide summary of the FE idea at the end



- we will use Firedrake (firedrakeproject.org) for the FE method
 - an open-source Python library for the solution of partial differential equations
- to use it one needs to know a key step: **write a weak form**

finite elements ... with no details

- by-hand solution of the Stokes model is impossible for most geometries
 - *exercise* on the exception: solve slab-on-a-slope
- so we solve numerically using the finite element (FE) method
 - this tutorial **suppresses the details** of FE
 - try e.g. Elman, Sylvester & Wathen (2014) for FE self-study
 - ▶ details one-slide summary of the FE idea at the end



- we will use Firedrake (firedrakeproject.org) for the FE method
 - an open-source Python library for the solution of partial differential equations
- to use it one needs to know a key step: **write a weak form**

the weak form

- start from power-law Stokes equations, now called the *strong form*:

$$-\nabla \cdot (2\nu_\epsilon(|D\mathbf{u}|) D\mathbf{u}) + \nabla p = \rho \mathbf{g} \quad (1)$$

$$\nabla \cdot \mathbf{u} = 0 \quad (2)$$

- multiply (1) by test velocity $\mathbf{v}$ and (2) by test pressure q , integrate by parts, and combine into one **nonlinear residual function**:

$$F(\mathbf{u}, p; \mathbf{v}, q) = \int_{\Lambda} 2\nu_\epsilon(|D\mathbf{u}|) D\mathbf{u} : D\mathbf{v} - p \nabla \cdot \mathbf{v} - q \nabla \cdot \mathbf{u} - \rho \mathbf{g} \cdot \mathbf{v} \, dx$$

- Λ is the spatial domain of ice
 - “ dx ” = $dx \, dy \, dz$ in 3D, and “ dx ” = $dx \, dz$ in the 2D planar case
- the actual **weak-form problem statement** is “residual = 0”:

$$F(\mathbf{u}, p; \mathbf{v}, q) = 0 \quad \text{for all test functions } \mathbf{v} \text{ and } q$$

- the FE method solves this problem over a finite-dimensional space

- nonlinear residual function:

$$F(\mathbf{u}, p; \mathbf{v}, q) = \int_{\Lambda} 2\nu_{\epsilon}(|D\mathbf{u}|) D\mathbf{u} : D\mathbf{v} - p\nabla \cdot \mathbf{v} - q\nabla \cdot \mathbf{u} - \rho \mathbf{g} \cdot \mathbf{v} \, dx$$

$$\text{with } \nu_{\epsilon}(|D\mathbf{u}|) = \frac{1}{2} B_n (|D\mathbf{u}|^2 + (\epsilon D_0)^2)^{((1/n)-1)/2}$$

- we write this using the *Unified Form Language* (UFL) in Firedrake:

```
def D(w):  
    return 0.5 * (grad(w) + grad(w).T)  
  
Du2 = 0.5 * inner(D(u), D(u)) + (eps * Dtyp)**2.0  
nu = 0.5 * Bn * Du2**((1.0 / n - 1.0)/2.0)  
  
fbody = rho * Constant((0.0, 0.0, - g))  
F = ( 2.0 * nu * inner(D(u), D(v)) \  
      - p * div(v) - q * div(u) - inner(fbody, v) ) * dx
```

- Python UFL code is quite similar to the math

mixed finite elements for fluid problems

- other details we need to know about:
 1. choose **function spaces** for the velocity and the pressure
 2. choose finite element spaces that (the experts say) are **stable**
- Firedrake makes choosing such **mixed elements** easy:

```
V = VectorFunctionSpace(mesh, 'CG', 2) # u space is P2
W = FunctionSpace(mesh, 'CG', 1)       # p space is P1
Z = V * W                               # mixed space
up = Function(Z)
u, p = split(up)
v, q = TestFunctions(Z)
```

- this $P_2 \times P_1$ choice is called Taylor-Hood
- other mixed spaces are just as easy
- e.g. we choose $P_2 \times DQ_1$ in stage4/, for reasons ...

<i>theory</i>	equations for the glacier dynamics problem
stage1/	first demo: linear Stokes
stage2/	power-law Stokes for glacier ice
stage3/	practical power: extruded meshes and 3D glaciers
<i>theory</i>	evolving glaciers: free-surface equation coupled to Stokes
stage4/	moving-margin dome solutions
	summary and learning more

the linear Stokes equations

- if we make viscosity constant ($\nu_0 > 0$) then we get a **linear** Stokes system:

$$\begin{aligned} -\nabla \cdot (2\nu_0 D\mathbf{u}) + \nabla p &= \rho \mathbf{g} \\ \nabla \cdot \mathbf{u} &= 0 \end{aligned}$$

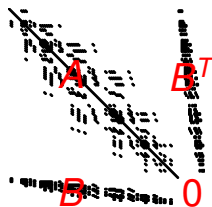
- classical reformat with $2\nabla \cdot D\mathbf{u} = \nabla^2 \mathbf{u}$:

$$\begin{aligned} -\nu_0 \nabla^2 \mathbf{u} + \nabla p &= \rho \mathbf{g} \\ -\nabla \cdot \mathbf{u} &= 0 \end{aligned}$$

- which has symmetric block structure:

$$\begin{bmatrix} A & B^\top \\ B & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ p \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ 0 \end{bmatrix}$$

- $A = -\nu_0 \nabla^2 \geq 0$
- $B^\top = (-\nabla \cdot)^\top = \nabla \dots$ from integration-by-parts



running the demonstration programs

- let's run a demo
- install Firedrake following these directions:
firedrakeproject.org/install.html
- activate the Python virtual environment every time you use Firedrake:

```
$ source ~/venv-firedrake/bin/activate
```

- other tools you will need (install from packages?):
 - Gmsh gmsh.info
 - Paraview paraview.org
- clone this tutorial, for both codes and slides:

```
$ git clone https://github.com/bueler/stokes-ice-tutorial.git
```



- *purpose*: solve linear Stokes on a trapezoidal “ice sheet”
- *source files*: `trap.geo`, `solve.py`
- *generated files*: `trap.msh`, `trap.pvd`

← inspect these

► details

these short codes are on slides at the end

```
$ cd stage1/  
$ gmsh -2 trap.geo           # mesh the domain  
$ gmsh trap.msh              # view the mesh  
$ python3 solve.py           # solve linear Stokes  
$ paraview trap.pvd          # view the solution
```



speed $|u|$

<i>theory</i>	equations for the glacier dynamics problem
<i>stage1/</i>	first demo: linear Stokes
<i>stage2/</i>	power-law Stokes for glacier ice
<i>stage3/</i>	practical power: extruded meshes and 3D glaciers
<i>theory</i>	evolving glaciers: free-surface equation coupled to Stokes
<i>stage4/</i>	moving-margin dome solutions
	summary and learning more

Newton's method

- linear Stokes in `stage1/` solved a single linear system:

$$\begin{bmatrix} A & B^\top \\ B & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ p \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ 0 \end{bmatrix}$$

- default solver: Firedrake $\rightarrow$ PETSc KSP $\rightarrow$ MUMPS
- but power-law Stokes residual function is **nonlinear in $\mathbf{u}$** :

$$F(\mathbf{u}, p; \mathbf{v}, q) = \int_{\Lambda} 2 \nu_{\epsilon}(|D\mathbf{u}|) D\mathbf{u} : D\mathbf{v} - p \nabla \cdot \mathbf{v} - q \nabla \cdot \mathbf{u} - \rho \mathbf{g} \cdot \mathbf{v} \, dx$$

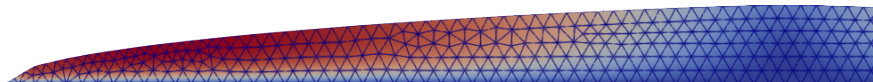
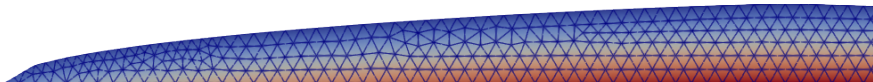
- fortunately, Firedrake applies Newton's method by default!
 - just write the problem as `solve(F == 0, ...)`
- what you need to know about Newton's iteration:
 - linearize (and solve a linear system) at each iterate
 - Firedrake uses UFL symbolic differentiation for linearization
 - Firedrake asks PETSc's SNES to do Newton; SNES options monitor and control the iteration



Isaac Newton

- *purpose*: solve the power-law Stokes equations on a dome-shaped glacier (ice sheet) with a flat bed
- *source files*: `domain.py`, `solve.py` ← inspect these
- *generated files*: `dome.geo`, `dome.msh`, `dome.pvd`

```
$ cd stage2/  
$ python3 domain.py      # generate domain outline  
$ gmsh -2 dome.geo       # mesh the domain  
$ python3 solve.py       # solve power-law Stokes  
$ paraview dome.pvd      # view the solution
```

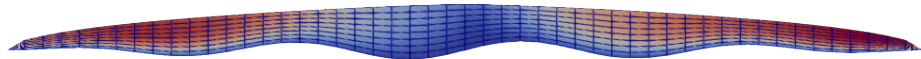
speed $|u|$ pressure p

<i>theory</i>	equations for the glacier dynamics problem
<i>stage1/</i>	first demo: linear Stokes
<i>stage2/</i>	power-law Stokes for glacier ice
<i>stage3/</i>	practical power: extruded meshes and 3D glaciers
<i>theory</i>	evolving glaciers: free-surface equation coupled to Stokes
<i>stage4/</i>	moving-margin dome solutions
	summary and learning more

a practical glacier solver

- let's use more practical and powerful approaches to meshing and solving
- we demonstrate (mostly) **dimension-independent programming**
 - code for 2D and 3D glacier geometry is nearly the same
 - dimension chosen at runtime
- we switch to an **extruded mesh**, and allow a bumpy bed
 - the **base mesh** has dimension-minus-one; it could come via Gmsh
 - now elements are quadrilaterals (in planar case), vs triangles in `stage2/`
 - elements are prisms in 3D
- additional bells & whistles:
 - rescaling the equations (optional)
 - vertical grid sequencing, with over-regularization on coarse meshes
 - additional diagnostic fields in the output `.pvd` file

solution speed $|\mathbf{u}|$ for default run:



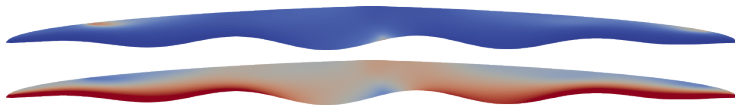
- *purpose*: a practical and robust solver
 - construct mesh by vertically-extruding a base mesh
 - vertical grid sequencing
 - write ν_ϵ , τ , $p - p_{\text{hydrostatic}}$ into output file
- *source file*: `solve.py`
- *generated file*: `dome.pvd`

*practical geometry
better initial iterates
better diagnostics*

← inspect this

```
$ cd stage3/  
$ python3 solve.py # default resolution  
$ python3 solve.py -mx 400 -refine 2 # higher resolution  
$ paraview dome.pvd
```

effective viscosity ν_ϵ and deviatoric stress magnitude $|\tau|$:



evidence of convergence

- in this stage we are setting resolution at runtime, for example:

```
$ python3 solve.py -b0 0 -mx 400 -mz 4 -refine 2
```

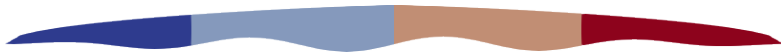
- using optional flat bed (`-b0 0`), to compare results to `stage2/`
 - initial mesh has 400 elements in horizontal and 4 in vertical
 - refine the vertical twice (by factors of four), to generate a 400×64 mesh
- velocity results, in meters per year, from a refinement study:

mesh	$\Delta x \times \Delta z$ (m)	average $ \mathbf{u} $	maximum $ \mathbf{u} $
50×8	400×125	1782	3263
100×16	200×63	1766	3211
200×32	100×31	1761	3196
400×64	50×16	1759	3191
800×128	25×8	1759	3190

- velocities are apparently converging
 - (we do not know the exact solution, and this is not verification)

parallel computation

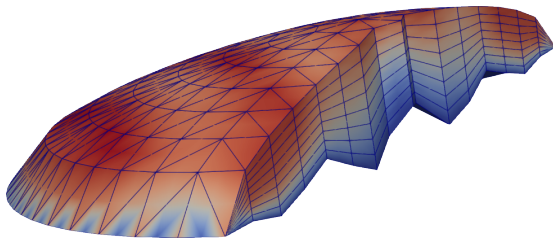
- **Firedrake codes work in parallel** (in normal usage)
 - extruded mesh gets partitioned following base mesh partition
 - see MPI `rank` field in `.pvd` output; result with 4 processes:



- with default direct solver (Mumps), results are essentially independent of number of processes
 - however, speed-up is sublinear because of significant serial workloads
- ```
$ python3 solve.py -mx 800 -refine 2 # 38 secs run time
```
- ```
$ mpiexec -n 4 python3 solve.py -mx 800 -refine 2 # 20 secs run time
```
- your performance results may vary ...

- *purpose*: demonstrate a (mostly) dimension-independent Firedrake code, to model a 3D glacier
- *source file*: `solve.py` ← inspect this
- *generated file*: `dome.pvd`

```
$ python3 solve.py -dim 3 -baserefine 3 -refine 1  
$ paraview dome.pvd
```



3× vertical exaggeration, coloring by speed $|\mathbf{u}|$

performance limitations of *this* solver

- this run is about as far as I can refine on my office desktop:

```
$ mpiexec -n 12 python3 solve.py -dim 3 -baserefine 4 \  
  -refine 2 -o finest.pvd
```
- *limiting factor*: our default **direct solver** for each Newton-step (linear) system uses too much memory when generating the LU factors
 - direct solvers experience *fill-in*, especially in 3D (B. 2021b)
- for higher resolution we need a **scalable solver**
 - built by composition of features: matrix-free, Schur-complement preconditioning, multigrid on the $\mathbf{u}-\mathbf{u}$ block
 - for example see Isaac, Stadler, & Ghattas (2015)
 - addressing surface-Stokes scalability properly is a different talk (B. 2023)

performance limitations of *this* solver

- this run is about as far as I can refine on my office desktop:

```
$ mpiexec -n 12 python3 solve.py -dim 3 -baserefine 4 \  
  -refine 2 -o finest.pvd
```
- *limiting factor*: our default **direct solver** for each Newton-step (linear) system uses too much memory when generating the LU factors
 - direct solvers experience *fill-in*, especially in 3D (B. 2021b)
- for higher resolution we need a **scalable solver**
 - built by composition of features: matrix-free, Schur-complement preconditioning, multigrid on the **u-u** block
 - for example see Isaac, Stadler, & Ghattas (2015)
 - addressing surface-Stokes scalability properly is a different talk (B. 2023)

<i>theory</i>	equations for the glacier dynamics problem
stage1/	first demo: linear Stokes
stage2/	power-law Stokes for glacier ice
stage3/	practical power: extruded meshes and 3D glaciers
<i>theory</i>	evolving glaciers: free-surface equation coupled to Stokes
stage4/	moving-margin dome solutions
	summary and learning more

don't glaciers actually change with the climate?

- we do not yet have an evolving glacier model
 - time derivatives have not even appeared!
 - the climate (surface mass balance) has not appeared as an input

Q. how do glaciers change shape?

A. via **free-surface equation**, a kinematical and conservation statement:

$$\frac{\partial s}{\partial t} - \mathbf{u}|_s \cdot \mathbf{n}_s = a \quad (*)$$

- $z = s(t, x, y)$ is the glacier surface elevation; initial $s(0, x, y)$ is needed
- $\mathbf{u}|_s$ is the surface value (*trace*) of the ice flow velocity
- $\mathbf{n}_s = \left\langle -\frac{\partial s}{\partial x}, -\frac{\partial s}{\partial y}, 1 \right\rangle$ is an upward surface-normal vector
- $a(t, x, y)$ is the **surface mass balance**, the source term
- $(*)$ holds on $(0, T) \times \Omega$, where $\Omega \subset \mathbb{R}^d$ is a fixed map-plane region

► details

we can also use ice thickness $H = s - b$ here, but s has advantages

don't glaciers actually change with the climate?

- we do not yet have an evolving glacier model
 - time derivatives have not even appeared!
 - the climate (surface mass balance) has not appeared as an input

Q. how do glaciers change shape?

A. via **free-surface equation**, a kinematical and conservation statement:

$$\frac{\partial s}{\partial t} - \mathbf{u}|_s \cdot \mathbf{n}_s = a \quad (*)$$

- $z = s(t, x, y)$ is the glacier surface elevation; initial $s(0, x, y)$ is needed
- $\mathbf{u}|_s$ is the surface value (*trace*) of the ice flow velocity
- $\mathbf{n}_s = \left\langle -\frac{\partial s}{\partial x}, -\frac{\partial s}{\partial y}, 1 \right\rangle$ is an upward surface-normal vector
- $a(t, x, y)$ is the **surface mass balance**, the source term
- $(*)$ holds on $(0, T) \times \Omega$, where $\Omega \subset \mathbb{R}^d$ is a fixed map-plane region

► details

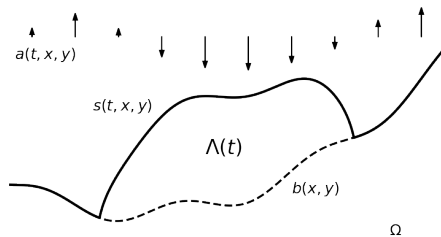
we can also use ice thickness $H = s - b$ here, but s has advantages

coupled equations

coupled free-surface & Stokes equations

$$\begin{aligned}\frac{\partial s}{\partial t} - \mathbf{u}|_s \cdot \mathbf{n}_s &= a && \text{free-surface equation} \\ -\nabla \cdot (2\nu_\epsilon D\mathbf{u}) + \nabla p &= \rho \mathbf{g} && \text{stress balance} \\ \nabla \cdot \mathbf{u} &= 0 && \text{incompressibility}\end{aligned}$$

- 3 solution variables: s , $\mathbf{u}$, p
- 2 coupling directions:
 - I. the free-surface equation has $\mathbf{u}|_s$ as a coefficient
 - II. $s(t, x, y)$ determines the domain $\Lambda(t)$ on which the Stokes sub-problem is solved, at each t



weak form of coupled system of equations

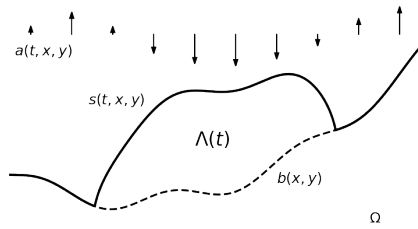
- we get the weak form of the coupled system in the usual way
- multiply by test functions and integrate:

$$\int_{\Omega} \left(\frac{\partial s}{\partial t} - \mathbf{u}|_s \cdot \mathbf{n}_s - a \right) r \, dx = 0 \quad \forall r$$

$$\int_{\Lambda(t)} 2 \nu_{\epsilon} (|D\mathbf{u}|) D\mathbf{u} : D\mathbf{v} - p \nabla \cdot \mathbf{v} - \rho \mathbf{g} \cdot \mathbf{v} \, dx = 0 \quad \forall \mathbf{v}$$

$$\int_{\Lambda(t)} -(\nabla \cdot \mathbf{u}) q \, dx = 0 \quad \forall q$$

- high-level choices for system:
 1. spaces and elements for $s, \mathbf{u}, p$?
 2. time-stepping: explicit or implicit?
 3. solved together (*monolithic*) or split?



challenges in solving the coupled system

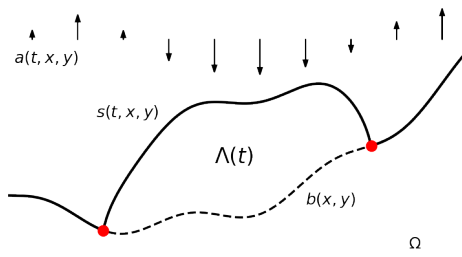
- early/naive approaches to solving this coupled problem have been fragile
- the method everyone tries first is to solve power-law Stokes for the current geometry, then advance the surface, and repeat
 - this is an *explicit* approach, which *splits* the coupling
 - resulting performance is **stability-limited**, in a confusing manner
- what are the underlying challenges?
- ① mesh geometry for the $\mathbf{u}, p$ solver changes at every time step
 - makes monolithic and/or implicit implementations difficult
- ② coupled problem mixes advective and diffusive aspects
 - proportions depend on stress boundary conditions
 - time-step restrictions and performance scaling remain unclear (B 2023)
 - but `stage4/` applies recent theoretical progress!

challenges in solving the coupled system

- early/naive approaches to solving this coupled problem have been fragile
 - the method everyone tries first is to solve power-law Stokes for the current geometry, then advance the surface, and repeat
 - this is an *explicit* approach, which *splits* the coupling
 - resulting performance is **stability-limited**, in a confusing manner
 - what are the underlying challenges?
- 1 mesh geometry for the $\mathbf{u}, p$ solver changes at every time step
 - makes monolithic and/or implicit implementations difficult
 - 2 coupled problem mixes advective and diffusive aspects
 - proportions depend on stress boundary conditions
 - time-step restrictions and performance scaling remain unclear (B 2023)
 - but `stage4/` applies recent theoretical progress!

also, glaciers have moving margins!

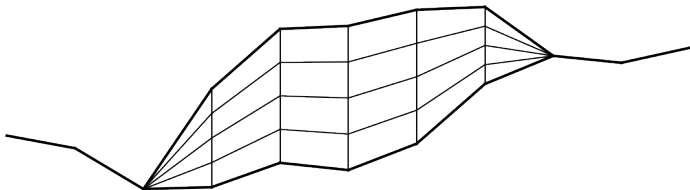
- the **surface elevation must be above the bed**
- enforcing this obvious fact provides the (lateral) **free-boundary condition** for a land-based glacier
 - the weak form is a **variational inequality** (B. 2021a, 2021b)
 - the thickness and horizontal flux are both zero at the free boundary
- solving the coupled problem subject to this condition determines the location of the moving glacier margin (free boundary)



<i>theory</i>	equations for the glacier dynamics problem
<i>stage1/</i>	first demo: linear Stokes
<i>stage2/</i>	power-law Stokes for glacier ice
<i>stage3/</i>	practical power: extruded meshes and 3D glaciers
<i>theory</i>	evolving glaciers: free-surface equation coupled to Stokes
<i>stage4/</i>	moving-margin dome solutions
	summary and learning more

evolving extruded glacier meshes

- we will demonstrate and test a 2D solver which is **semi-implicit**, **split**, **stabilized**, and **moving-margin**
 - ... as explained on the next few slides
 - caution: “stabilized” will mean several things!
- using an extruded mesh between continuous P_1 (piecewise-linear) surfaces $s(t, x)$ and $b(t, x)$:
 - the Stokes domain $\Lambda(t)$ is **polygonal** (in space) at each t
 - the elements are quadrilaterals (in planar 2D)



- we use **new techniques** which modify the coupled weak form

3 techniques

1. first-order, and mostly explicit, time-stepping
 - A. with an implicit **edge stabilization** in the free-surface equation
 - B. and a CFL condition when margins are moving
 2. split, with alternating free-surface and Stokes solves
 - A. with a **load stabilization** coupling to the Stokes solver

[▶ details](#) connection to free-surface mantle flow
 3. the coupled free-surface problem is solved as a **variational inequality** which determines the glacier extent and margin
 - A. with a bounds-respecting Newton method
 - B. and “pinching” for variable ice extent on the extruded mesh
- the next slides give citations and details
 - techniques 1A and 2A are the **Stockholm stabilizations**

technique 1A: implicit edge stabilization in free-surface equation

- introduced for glaciers by Tominec et al (2026)
 - from Burman & Fernández (2009) and Ern & Guermond (2013)
- notation: s_k^h is FE surface elevation at time t_k
- add (semi-)implicit and weighted FE **edge stabilization**:

$$\int_{\Omega^h} \left(\frac{s_{k+1}^h - s_k^h}{\Delta t} - \mathbf{u}|_{s_k^h} \cdot \mathbf{n}_{s_k^h} - a \right) r^h dx + J(s_{k+1}^h, r^h) = 0$$

for all test functions r^h , where

$$J(s, r) = \sum_{e \in E^h} \int_e \frac{1}{2} h_e^2 \left| \mathbf{u}|_{s_k^h} \right| \llbracket \nabla s \rrbracket \llbracket \nabla r \rrbracket dS$$

- E^h is the set of interior edges in the triangulation Ω^h
- $\llbracket \mathbf{v} \rrbracket = \mathbf{v}_+ \cdot \mathbf{n}_+ - \mathbf{v}_- \cdot \mathbf{n}_-$ is a **vector field jump along an edge**
- h_e is the average of the diameters of the two cells which meet e
- surface velocity $(\mathbf{u}|_{s_k^h})$ and slope $(\mathbf{n}_{s_k^h})$ are evaluated **explicitly**, at t_k
- we solve a **linear** system for unknown s_{k+1}^h



Igor Tominec



Lukas Lundgren



André Löfgren



Josefin Ahlkrone

technique 1B: CFL determination of time step

- our application adds an advancing glacier margin, which is not considered in the stability proof from Tominec et al (2026)
- because edge stabilization technique 1A is a splitting, with **velocity evaluated explicitly**, margin advance is limited to one base-mesh element per time step
- time step Δt therefore limited by CFL criterion:

$$\Delta t = C_{\text{CFL}} \min \frac{h}{|\mathbf{u}|}$$

- we get reasonable results with $C_{\text{CFL}} = 0.25$, like Ern & Guermond (2013)

technique 2A: semi-implicit coupling heuristic in stress balance

- from Löfgren et al (2022); refined by Tominec et al (2026)
- add **extrapolated surface mass balance load** to Stokes form:

$$\int_{\Lambda(t_k)} 2\nu_\epsilon(|D\mathbf{u}|) D\mathbf{u} : D\mathbf{v} - p\nabla \cdot \mathbf{v} - \rho\mathbf{g} \cdot \mathbf{v} \, dx \\ + \int_{\Gamma_s(t_k)} \rho g \Delta t \left[\frac{1}{2}(\mathbf{u}|_{s_k} \cdot \mathbf{n}_{s_k}) + a \right] (\mathbf{v} \cdot \mathbf{n}) \, ds = 0$$

- $\Lambda(t_k)$ is the Stokes domain defined by the current surface elevation s_k
 - $\Gamma_s(t_k)$ is the upper surface of $\Lambda(t_k)$
 - $\mathbf{n}_{s_k} = \langle -\nabla s_k, 1 \rangle$ is an outward normal on $\Gamma_s(t_k)$
 - $\mathbf{n} = \mathbf{n}_{s_k} / |\mathbf{n}_{s_k}|$ is the outward *unit* normal
- concept: modify the stress balance to stabilize the coupled solve
 - Reynold's transport theorem motivates this (symmetrized) extrapolation
 - ... which puts time step Δt and SMB a inside the Stokes solve!



André Löfgren



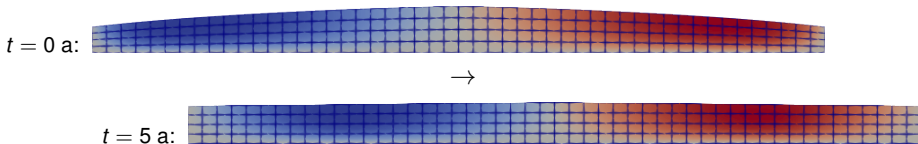
Josefin Ahlkrone



Christian Helanow

- *purpose*: exhibit unconditional **numerical stability** and **exact mass conservation** for an evolving glacier between lubricated walls
 - geometry considered by Löfgren (2022) and Tominec (2026)
 - Tominec (2026) **prove** that the L^2 norm of the evolving surface can't blow up
- *source files*: `geometry.py`, `halfar.py`, `solve.py` ← **inspect these**
- *generated files*: `dome.pvd`, `movie.pvd`

```
$ cd stage4/  
$ python3 solve.py -walls -L 8000 -omovie movie.pvd  
$ paraview movie.pvd
```



instability showing need for stabilization technique 2A

- *without* technique 2A, the maximum size of the stable time step is significantly restricted (Löfgren 2022)
 - many time step choices produce “sloshing” and Stokes solver failures
 - e.g. runs to produce table:

```
$ python3 solve.py -walls -L 8000  
-noload -nocfl -mx MX -maxdt DT
```


with $MX = 25, 50, 100, 200, 400$
- to demonstrate what happens *without* technique 2A, for each resolution in this table we started with $\Delta t = 1$ a, and reduced Δt by a factor of two until the 5 a run completes without a solver failure

Δx (m)	stable Δt (a) w/o technique 2A
640	0.25
320	0.25
160	0.125
80	0.125
40	0.125

- Tominec (2026) do additional experimentation
- **with technique 2A, arbitrary time steps are stable**, and exhibit no sloshing

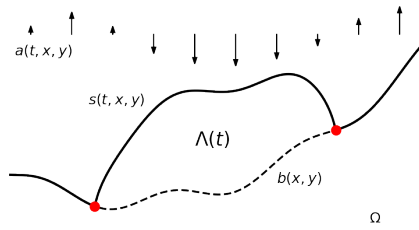
technique 3A: semi-implicit variational inequality for free surface

- define the FE **admissible set** $\mathcal{K}^h = \{r^h \in P_1 : r^h \geq b^h\}$
 - the P_1 bed elevation function b^h is time-independent
- the new surface $s_{k+1}^h \in \mathcal{K}^h$ solves this variational inequality weak form:

$$\int_{\Omega} \left(\frac{s_{k+1}^h - s_k^h}{\Delta t} - \mathbf{u}|_{s_k^h} \cdot \mathbf{n}_{s_k^h} - a \right) (r^h - s_{k+1}^h) dx + J(s_{k+1}^h, r^h - s_{k+1}^h) \geq 0$$

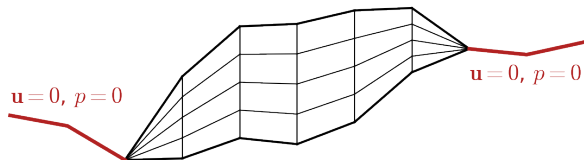
for all r^h in $\mathcal{K}^h$

- includes edge stabilization
- velocity $\mathbf{u}|_{s_k^h}$ still explicit**
- the computed margin location is where s_{k+1}^h meets b^h
- we apply the `vinewtonrsls` bounds-respecting Newton solver from PETSc (B. 2021b)



technique 3B: extruded mesh with variable ice extent

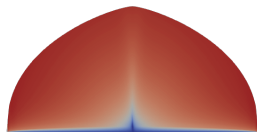
- the base mesh covers all of Ω , including areas with no ice
- the extruded mesh has a fixed number m_z of levels in the vertical
 - $\mathbf{u}, p$ are defined at each node of the extruded mesh
 - our meshes and fields have **constant memory footprint** as we time-step
- in columns with no ice we lock/pinch $\mathbf{u}, p$ values to zero



- s_k^h, b^h define ice domain $\Lambda(t_k)$ in this semi-implicit framework
- $s_k^h = b^h$ at a base mesh node $\implies$ no ice in that extruded column
- PinchColumnPressure|Velocity sub-class DirichletBC
- [details](#) for p , switch from P_1 to $DQ1$ “spectral” to avoid bad pinching

- *purpose*: moving-margin free surface, exhibiting numerical stability
- *source files*: `geometry.py`, `halfar.py`, `solve.py` ← inspect these
- *generated files*: `dome.pvd`, `movie.pvd`

```
$ python3 solve.py -mx 200 -mz 10 -T 100 -L 20000 \  
    -omovie movie.pvd  
$ paraview movie.pvd
```

 $t = 0$ a $t = 100$ a

10× vertical exaggeration, $\log(|\mathbf{u}|)$ coloring

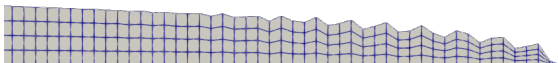
instability showing need for stabilization technique 1A

- especially near the glacier margin, **edge stabilization** (technique 1A) is **clearly needed**
- compare these runs:

```
$ python3 solve.py -mx 200 -T 1
```



```
$ python3 solve.py -mx 200 -T 1 -noedge
```



- runs with lubricated walls generally do not show the need

instability showing need for stabilization technique 1B

- need for technique 1B (CFL for time-step length) is more subtle
- compare the margin shape at the end of these two runs:

```
$ python3 solve.py -mx 100
```



```
$ python3 solve.py -mx 100 -nocfl -maxdt 1.0
```



- advancing margin shape without CFL-limited time steps is wrong
 - bulging and not downsloping, but not obviously unstable
 - not close to Halfar margin location (next slides)

Q. is our time-dependent solver correct?

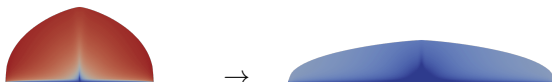
- it is not clear how to build a moving-margin, coupled Stokes-surface exact solution
- however, such exact solutions exist for **shallow ice approximation (SIA)**
- SIA exact solutions are **close** to solving the free-surface + Stokes coupled equations when the aspect ratio $\epsilon = (\text{height})/(\text{width})$ is small
- next slide: we compare with the Halfar (1981) SIA exact solution
 - on a **shallow initial dome geometry**, SIA is reasonably accurate
 - e.g. initial height 1000 m and width 20 km yields $\epsilon = 0.05$



Peter Halfar

evidence of correctness for a shallow, moving-margin dome

- consider runs of 10 years at different horizontal resolutions:
 - `$ python3 solve.py -mz 10 -T 10 -mx MX`
 - use $MX = 50, 100, 200, 400, 800$, giving $75 \leq \Delta x \leq 1200$ m
 - from 33 ($MX = 50$) to 557 ($MX = 800$) adaptive-scheme time steps
 - max computed ice speed 4600 m/a initially; 160 m/a for final state



- compare final computed geometry to Halfar's prediction:

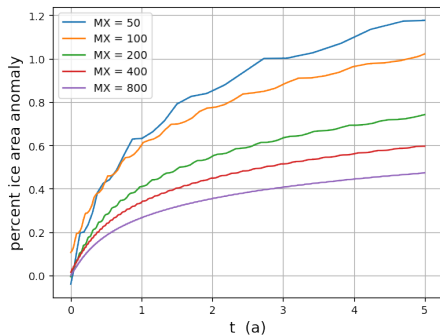
Δx (m)	width (km)	center height (m)
600	28.800	697.789
300	28.800	697.026
150	28.800	696.107
75	28.950	695.670
37.5	28.875	695.329
Halfar	28.993	689.829

- reasonably close!
- ... but we do *not* expect precise convergence to Halfar value

mass conservation errors with moving glacier margins

- since $a = 0$ here, mass is exactly conserved in the continuum model

- unfortunately, our scheme **does not conserve mass** here
- numerical mass (= ice area) drifts about 1% over 5 years
 - curves should be horizontal lines
 - higher res $\implies$ better conserve



- this is **only a moving margin effect**
 - our scheme conserves mass for fixed walls, as proved by Tominec 2026
 - here mass is non-decreasing; prove from VI using $a = 0$
 - this margin mass conservation issue is distinct from those in Jarosch et al (2013) and B (2021a)

Outline

- theory* equations for the glacier dynamics problem
- stage1/* first demo: linear Stokes
- stage2/* power-law Stokes for glacier ice
- stage3/* practical power: extruded meshes and 3D glaciers
- theory* evolving glaciers: free-surface equation coupled to Stokes
- stage4/* moving-margin dome solutions
- summary and learning more

summary and outlook

- Firedrake makes solving the glacier Stokes problem **easy for the new numerical modeler**:
 - the Python codes are short
 - UFL (weak forms) and PETSc (nonlinear solvers) are already in the stack
- **modern techniques work**:
 - extruded meshes (Firedrake), because glaciers are thin flows
 - Stockholm stabilizations, so time-stepping is safe and easy
 - Tominec et al (2026) has 1st coupled Stokes-surface *proof* in glaciology?
 - variational inequality solvers (PETSc) for physically-correct moving margins
- remaining improvements and challenges:
 - 1 make Stokes solver faster by replacing direct linear algebra with a **scalable solver** (matrix-free multigrid)
 - 2 make coupled free-surface-and-Stokes solver via a scalable **monolithic implicit step** (FASCD multigrid)
 - 3 make time-stepping glacier simulations **mass conserving even in free-boundary cases** (how?)

thanks for listening!

summary and outlook

- Firedrake makes solving the glacier Stokes problem **easy for the new numerical modeler**:
 - the Python codes are short
 - UFL (weak forms) and PETSc (nonlinear solvers) are already in the stack
- **modern techniques work**:
 - extruded meshes (Firedrake), because glaciers are thin flows
 - Stockholm stabilizations, so time-stepping is safe and easy
 - Tominec et al (2026) has 1st coupled Stokes-surface *proof* in glaciology?
 - variational inequality solvers (PETSc) for physically-correct moving margins
- remaining improvements and challenges:
 - 1 make Stokes solver faster by replacing direct linear algebra with a **scalable solver** (matrix-free multigrid)
 - 2 make coupled free-surface-and-Stokes solver via a scalable **monolithic implicit step** (FASCD multigrid)
 - 3 make time-stepping glacier simulations **mass conserving even in free-boundary cases** (how?)

thanks for listening!

github.com/bueler/stokes-ice-tutorial



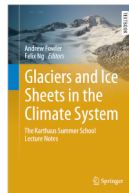
- Firedrake: firedrakeproject.org
- PETSc: petsc.org



Firedrake

 PETSc

- for power-law Stokes eqns, see Ch. 1 by Hewitt in Fowler & Ng (2021)
- for finite elements and Stokes: Elman, Silvester, & Wathen (2014)
- for PETSc, Firedrake, and Stokes: Bueler (2021b)



- E. Bueler (2021a). *Conservation laws for free-boundary fluid layers*, SIAM J. Appl. Math. 81 (5), 2007–2032 doi:[10.1137/20M135217X](https://doi.org/10.1137/20M135217X)
- E. Bueler (2021b). PETSc for Partial Differential Equations: Numerical Solutions in C and Python, SIAM Press
- E. Bueler (2023). *Performance analysis of high-resolution ice-sheet simulations*, J. Glaciol. 69 (276), 930–935 doi:[10.1017/jog.2022.113](https://doi.org/10.1017/jog.2022.113)
- E. Burman & M. A. Fernández (2009). *Finite element methods with symmetric stabilization for the transient convection-diffusion-reaction eqn*, CMAME 198, 2508–2519 doi:<https://doi.org/10.1016/j.cma.2009.02.011>
- H. Elman, D. Silvester, & A. Wathen (2014), Finite Elements and Fast Iterative Solvers, With Applications in Incompressible Fluid Dynamics, 2nd ed., Oxford University Press
- A. Ern & J.-L. Guermond (2013). *Weighting the edge stabilization*, SIAM J. Numer. Analysis 51 (3), 1655–1677 doi:[10.1137/120867482](https://doi.org/10.1137/120867482)
- A. Fowler & F. Ng, ed. (2015). *Glaciers and Ice Sheets in the Climate System: The Karthaus Summer School Lecture Notes*, Springer Press
- P. Halfar (1981). *On the dynamics of the ice sheets*, J. Geophys. Res. 86 (C11), 11065–11072
- T. Isaac, G. Stadler, & O. Ghattas (2015). *Solution of nonlinear Stokes equations discretized by high-order finite elements . . .*, SIAM J. Sci. Comput. 37 (6), B804–B833 doi:[10.1137/140974407](https://doi.org/10.1137/140974407)
- A. H. Jarosch, C. G. Schoof, & F. S. Anslow (2013). *Restoring mass conservation to shallow ice flow models over complex terrain*, Cryosphere 7 (1), 229–240 doi:[10.5194/tc-7-229-2013](https://doi.org/10.5194/tc-7-229-2013)
- B. Kaus, H. Mühlhaus, & D. May (2010). *A stabilization algorithm for geodynamic numerical simulations with a free surface*, Phys. Earth Planet. Inter. 181, 12–20 doi:[10.1016/j.pepi.2010.04.007](https://doi.org/10.1016/j.pepi.2010.04.007)
- A. Löfgren, J. Ahlkrone & C. Helanow (2022). *Increasing stable time-step sizes of the free-surface problem arising in ice-sheet simulations*, J. Comput. Phys.: X 16 (100114) doi:[10.1016/j.jcp.2022.100114](https://doi.org/10.1016/j.jcp.2022.100114)
- I. Tominec, L. Lundgren, A. Löfgren, & J. Ahlkrone (2026). *Free-surface Stokes problem: stability estimates and time-step improvements*, IMA J. Numer. Analysis (in press)

avoiding unbounded viscosity values

- the unmodified power-law viscosity formula yields unbounded values as $D\mathbf{u} \rightarrow 0$, i.e. for vanishing strain rates
 - note that $\nu = \frac{1}{2} B_n |D\mathbf{u}|^{(1/n)-1}$ has a negative power
- so we also regularize with **small** $\epsilon > 0$ to generate bounded viscosity

power-law Stokes equations with regularized viscosity

$$-\nabla \cdot (2\nu_\epsilon(|D\mathbf{u}|)D\mathbf{u}) + \nabla p = \rho \mathbf{g} \quad \text{stress balance}$$

$$\nabla \cdot \mathbf{u} = 0 \quad \text{incompressibility}$$

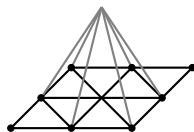
$$\nu_\epsilon(|D\mathbf{u}|) = \frac{1}{2} B_n (|D\mathbf{u}|^2 + (\epsilon D_0)^2)^{((1/n)-1)/2} \quad \text{flow law}$$

- we use $\epsilon = 10^{-4}$ and $D_0 = 1 \text{ a}^{-1}$ in the demonstrations

[◀ return](#)

how the finite element (FE) method works *(in just one slide!)*

- example PDE problem: $-\nabla^2 u = f$, $u = 0$ on $\partial\Omega$
- weak form: $\int_{\Omega} \nabla u \cdot \nabla v = \int_{\Omega} f v$ for all v
- build **hat functions** ψ_j on a mesh $\mathcal{T}_h \rightarrow$
- **test** functions are hats: $v = \psi_j$
- approximate solution also a sum of hats (**trial** function): $u_h = \sum_j u_j \psi_j$
- system **assembly**: element-by-element, compute and sum integrals to build a matrix A and vector $\mathbf{b}$



$$A_{ij} = \sum_{T \in \mathcal{T}_h} \int_T \nabla \psi_j \cdot \nabla \psi_i, \quad b_i = \sum_{T \in \mathcal{T}_h} \int_T f \psi_i$$

- hat functions are local $\implies$ few elements in a sum $\implies A$ is sparse
- **solve** for nodal value u_j : $A\mathbf{u} = \mathbf{b}$
 - use e.g. Gaussian elimination, and Newton if system is nonlinear
- Stokes is similar, but see Elman et al (2014) or B (2021b) for details

- this generates the domain outline, and labels sides so that boundary conditions can work
- generates trap.msh: `$ gmesh -2 trap.geo`

```
// hand-written trapezoid, the boundary of our domain

lc = 5000.0; // tell Gmsh we want 5km triangles
Point(1) = {0.0,0.0,0,lc};
Point(2) = {20000.0,0.0,0,lc};
Point(3) = {15000.0,1000.0,0,lc};
Point(4) = {5000.0,1000.0,0,lc};
Line(5) = {1,2};
Line(6) = {2,3};
Line(7) = {3,4};
Line(8) = {4,1};
Line Loop(9) = {5,6,7,8};
Plane Surface(10) = {9};

// add labels so solver knows where to apply boundary conditions
// boundary ids: 41, 42 = top, base
Physical Line(41) = {6,7,8};
Physical Line(42) = {5};
Physical Surface(51) = {10};
```

- this uses Firedrake to solve the problem
- generates Paraview-readable file trap.pvd: `$ python3 solve.py`

```
from firedrake import *

mesh = Mesh("trap.msh") # run "gmsh -2 trap.geo" to generate

V = VectorFunctionSpace(mesh, "CG", 2)
W = FunctionSpace(mesh, "CG", 1)
Z = V * W
up = Function(Z)
u, p = split(up)
v, q = TestFunctions(Z)

def D(w): # strain-rate tensor
    return 0.5 * (grad(w) + grad(w).T)

g = 9.81 # m s-2
rho = 910.0 # kg m-3
```

continued next slide

stage1/ code: solve.py cont.

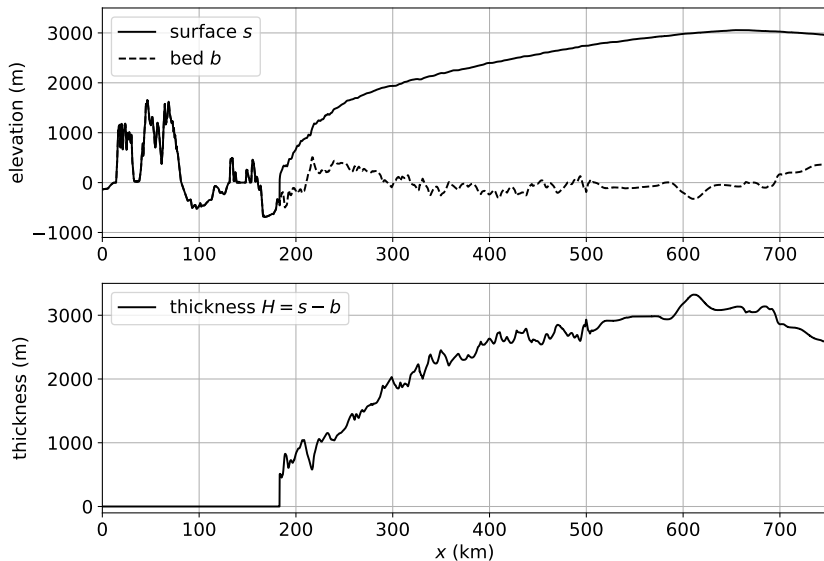
```
nu0 = 1.0e12 # Pa s
fbody = Constant((0.0, -rho * g))
F = 2.0 * nu0 * inner(D(u), D(v)) * dx \
    - (p * div(v) + q * div(u) + inner(fbody, v)) * dx
bcs = [DirichletBC(Z.sub(0), Constant((0.0, 0.0)), (42,))]

par = {
    "snes_type": "ksponly",
    "ksp_type": "preonly",
    "pc_type": "lu",
    "pc_factor_shift_type": "inblocks",
}
solve(F == 0, up, bcs=bcs, solver_parameters=par)

u = up.subfunctions[0]
p = up.subfunctions[1]
u.rename("velocity")
p.rename("pressure")
VTKFile("trap.pvd").write(u, p)
```

[◀ return](#)

geometry variable: surface elevation s versus ice thickness H



geodynamics connection: stabilization in free-surface mantle flow

- load-stabilization “technique 2A” builds on the **free surface stabilization approach** (FSSA) from Kaus et al (2010)

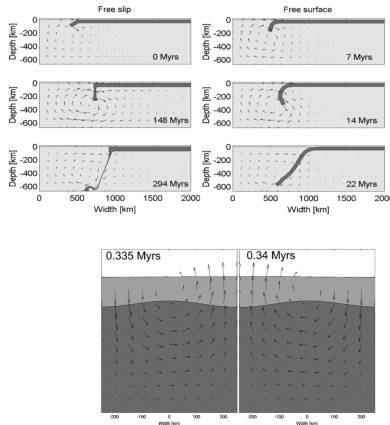
- viscous free-surface geodynamics

- FSSA adds extrapolated “boundary traction” term to Stokes stress balance:

$$\int_V (\sigma_{ij} w_{i,j} - \rho g_i w_i) - \int_\Gamma w_i (t_i + t_i^g) = 0$$

where $t_i^g = \Delta t \rho \nu_n g_i w_i$

- longer time steps by reducing sloshing →



- technique 2A is essentially FSSA + symmetrization (Tominec 2026)
- role for glaciers: essentially the same, but *without* (major) density contrasts and *with* a lateral free boundary

← return

pressure elements with interior degrees of freedom

- we do **not** want to pinch pressure p to zero anywhere in the closest-to-margin elements containing ice
 - doing so causes an unintended Dirichlet condition on the pressure
- therefore prefer pressure elements with **interior degrees of freedom**:

```
W = FunctionSpace(mesh, "DQ", 1, variant="spectral") # best
W = FunctionSpace(mesh, "DG", 0) # adequate
```

- for such elements the moving-margin mass conservation error (see slides) is provably one-sided

◀ return